Deep Learning

Team members:

- Cristina Postoronca
- Maisam Mohammadi
- Adis Murtezani
- Andreas Atzmillner

Installation and Preparation

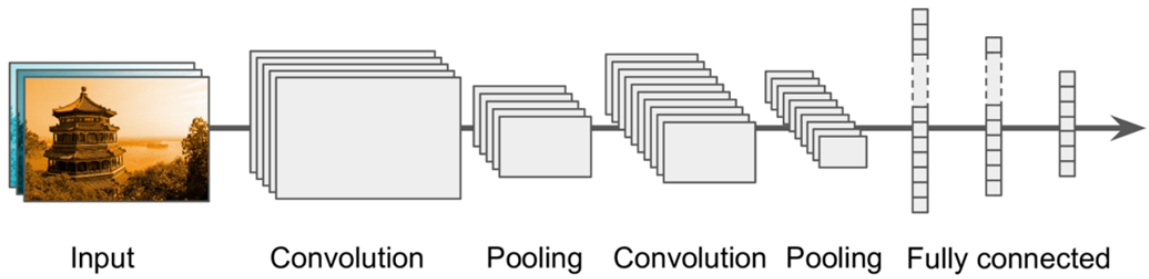
1. We install the environment in a directory (e.g. “deepLearning”) with the command:
    a. py-m venv .venv
2. Then we activate the environment with the command:
    a. .venv\Scripts\activate
3. Next, we install all required packages with the commands:
    a. pip install tensorflow
    b. pip install keras
    c. pip install matplotlib
    d. pip install numpy
    e. pip install scikit-learn

Topology

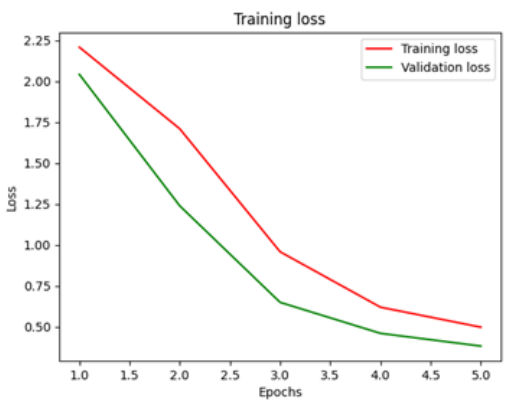
The function of **Convolutional Layers** is to extract visual features, using filters (also known kernels). Each filter is responsible for identifying a specific feature and it is sliding (so it's done on overlapping regions) across the image to look for the specific feature all over the image. At the core of convolutional layers are mathematical operations, that are performed at each position. As a result, a filter detects if the feature it is responsible for is present or not.

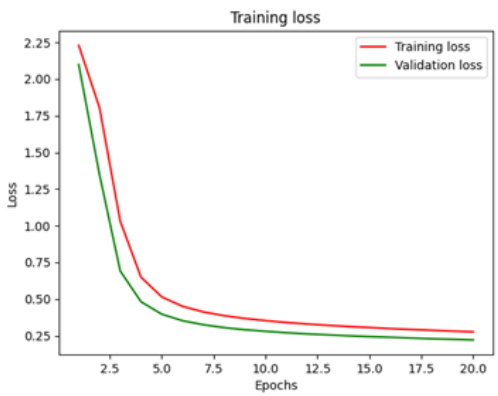
The features in the earlies layers are simpler: edges, corners, simple shapes. The further we go into the layers, the more complex the features: complex shapes, textures.

The function of **Pooling Layers** is to reduce the spatial dimensions of the images, so that computations are done faster and more efficient, but key is that they retain the most important information about the features. Similar to convolutional layers, pooling layers have a specific size, usually much smaller than the image size. But unlike the convolutional layers, the pooling is done on not overlapping in this case. For each non-overlapping area, the maximum values are taken (hence the max in MaxPooling). That results in a smaller output image. If the size is 2x2 then the output image is twice as small as the input image. So it reduces computational costs, but also makes the model more robust to slight variations, because it forces the model to learn more general features instead of the exact positions of pixels.

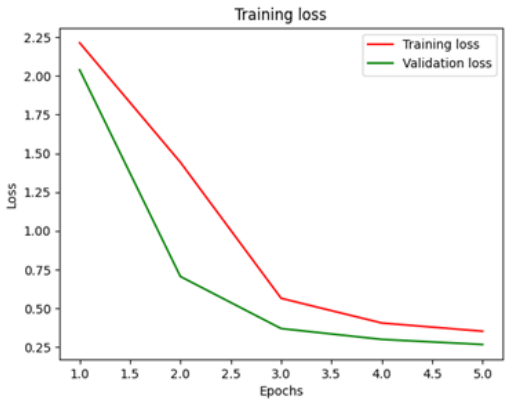


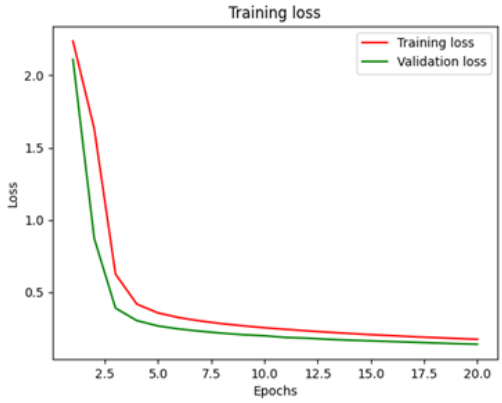
1 conv layer, 20 filters, 5 epochs vs 1 conv layers, 20 filters, 20epochs



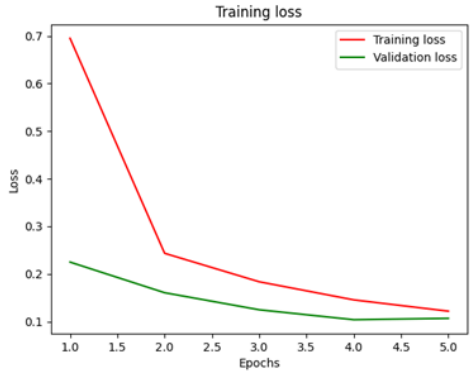


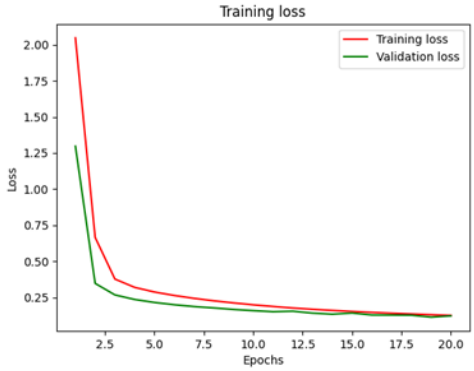
2 conv layers, 20 filters, 5 epochs vs 2 conv layers, 20 filters, 20 epochs



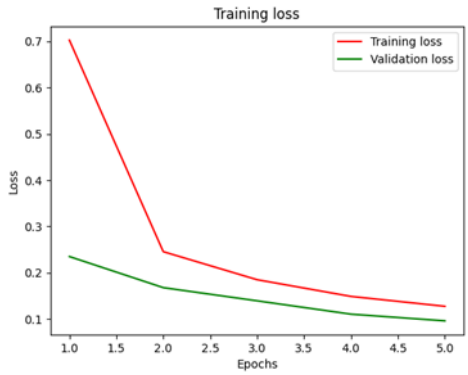


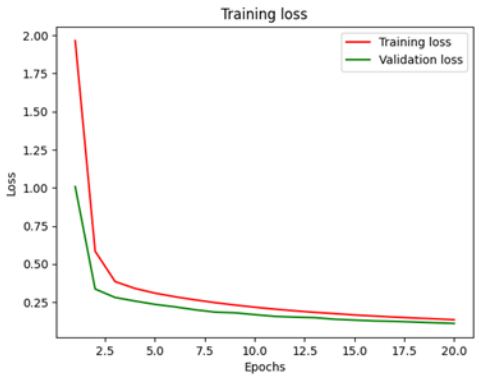
3 conv layers, 20 filters, 5 epochs vs 3 conv layers, 20 filters, 20 epochs



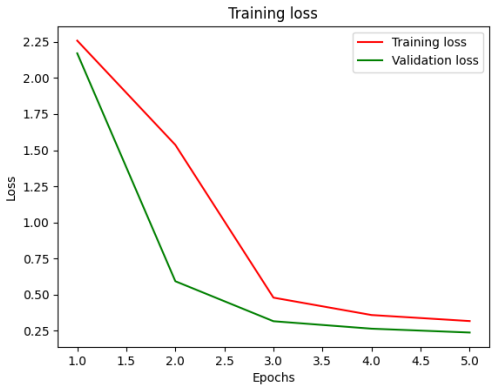


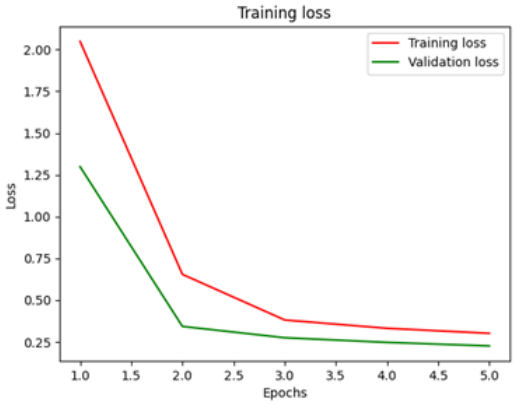
3 conv layers, 40 filters, 5 epochs vs 3 conv layers, 40 filters, 20 epochs



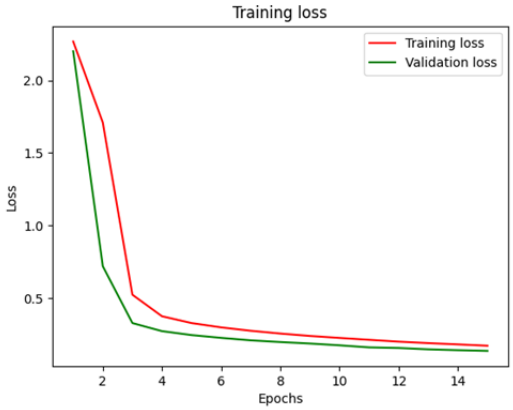


3 conv layers, 5 filters, 5 epochs vs 3 conv layers, 15 filters, 5 epochs





3 conv layers, 25 filters, 15 epochs



	Trained Models	accuracy	val_accuracy	loss	val_loss
1	1 layer 20 filters 5 epochs	0.8735	0.9073	0.4990	0.3836
2	1 layer 20 filters 20 epochs	0.9217	0.9367	0.2765	0.2212
3	2 layers 20 filters 5 epochs	0.8999	0.9255	0.3527	0.2672
4	2 layers 20 filters 20 epochs	0.9506	0.9635	0.1734	0.1394
5	3 layers 20 filters 5 epochs	0.9135	0.9365	0.3019	0.2246
6	3 layers 20 filters 20 epochs	0.9634	0.9685	0.1241	0.1213
7	3 layers 40 filters 5 epochs	0.9146	0.9372	0.2953	0.2199
8	3 layers 40 filters 20 epochs	0.9602	0.9690	0.1343	0.1101
9	3 layers 5 filters 5 epochs	0.9089	0.9327	0.3161	0.2365
10	3 layers 15 filters 5 epochs	0.9136	0.9383	0.3020	0.2266
11	3 layers 25 filters 15 epochs	0.9497	0.9637	0.1741	0.1384

Observations:

- One pair of conv + max pooling layers:

The weakest performance. Probably underfitting, because both training and validation loss values are relatively high (~0.45 for 5 epochs and ~0.25 for 20 epochs)

- Two pairs of conv + max pooling layers:

Better metric values compared to one pair. Loss is down to ~0.15, with 20 epochs, but still high with 5 epochs only. Accuracy is up to ~0.96 with 20 epochs, which is already very good. Conclusion, two pairs can produce good results if the number of epochs is on the higher side.

- Two pairs of conv + max pooling layers:

Best metrics so far. When using 5 epochs, loss is still high, so not optimal. The accuracy improves gradually with the nr of epochs and/or the nr of filters.

The best accuracy is achieved with 20 epochs and 40 filters ~ 0.96, same with the best loss value ~0.12. A slightly worse accuracy and loss is achieved by the same nr of epochs but twice as few filters (20), so probably from a resource point of view, this might be the optimal configuration for MNIST dataset.

Learning Rate

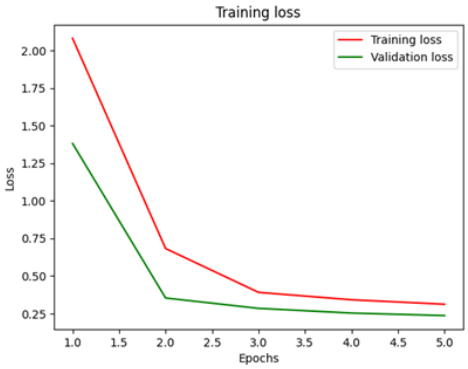
The **learning rate** controls the size of the step of the optimizer when tuning the weights. The goal is to go down the gradient, to the mimimal loss, as quickly and efficient as possible.

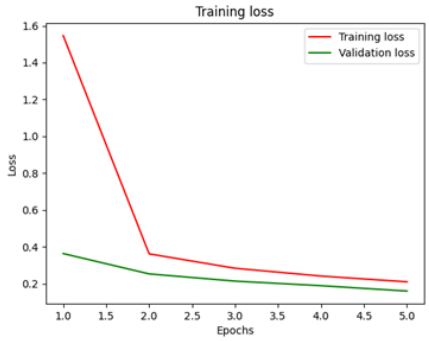
A step size that is too small will result in a slower training, it has a higher risk to get stuck in a local mimimum and is more demanding of the computational resources.

A step size that is too large will most probably result in unstable training. It oscilates too much, can overshoot the minimum and is not efficient in finding it.

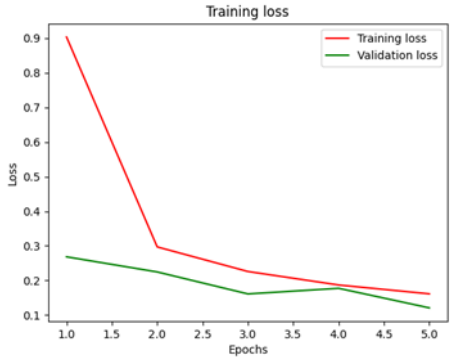
So to avoid these issues, a optimal step size needs to be found. It will result in a stable but also efficient training. It finds the minimum loss quickly and efficiently. The loss curve is smooth, don't usually have spikes and is not usually oscilating (unlike when the step is too big). So a correctly chosen step size will result in not getting stuck in a local minimum and will not fail to find the best solution.

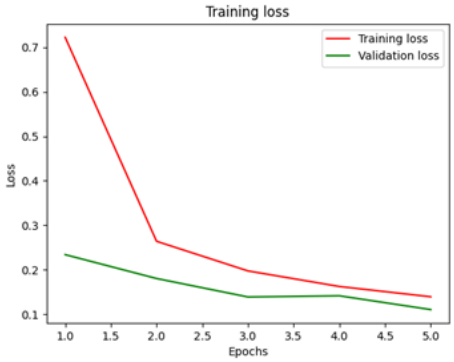
5 epochs lr: 0.001 vs 5 epochs lr: 0.0025



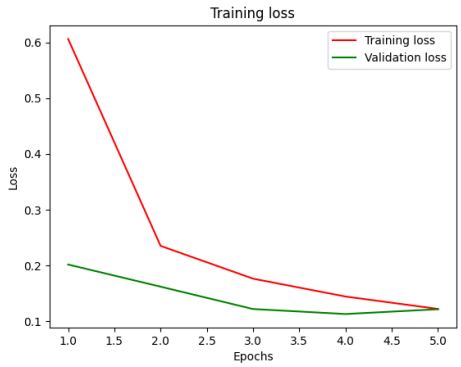


5 epochs lr: 0.005 vs 5 epochs lr: 0.0075

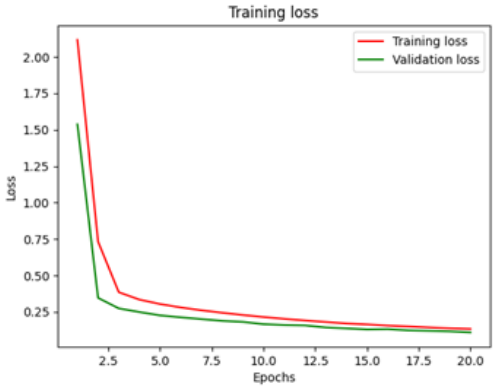


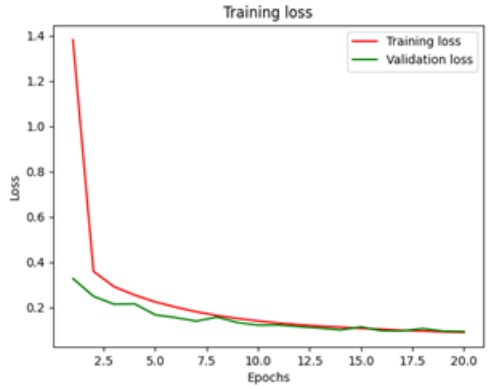


5 epochs lr: 0.01

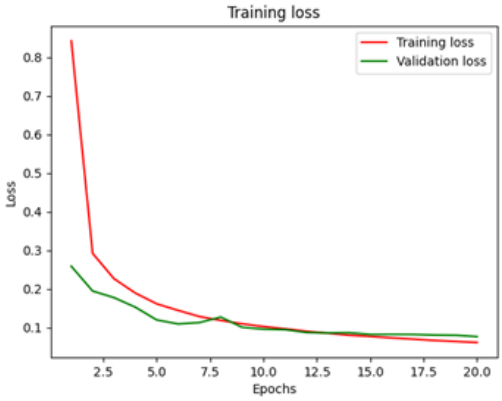


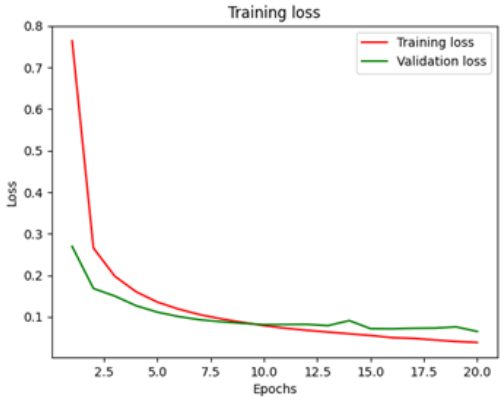
20 epochs lr: 0.001 vs 20 epochs lr: 0.0025



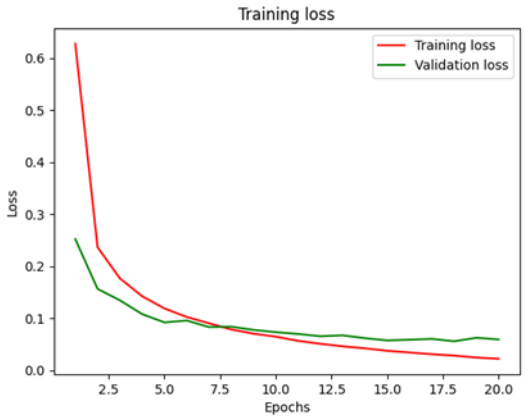


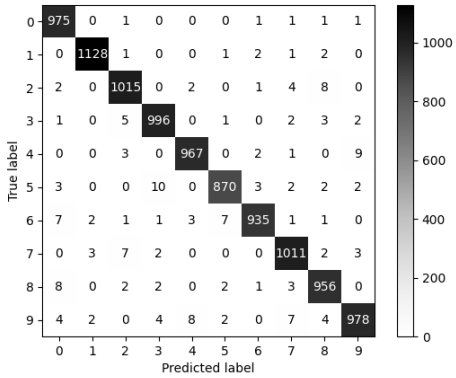
20 epochs lr: 0.005 vs 20 epochs lr: 0.0075





20 **epochs** lr: 0.01





	Trained Models	accuracy	loss	val_accuracy	val_loss
1	5 epochs lr: 0.001	0.9100	0.3124	0.9342	0.2369
2	5 epochs lr: 0.0025	0.9379	0.2097	0.9577	0.1596
3	5 epochs lr: 0.005	0.9527	0.1612	0.9683	0.1207
4	5 epochs lr: 0.0075	0.9590	0.1392	0.9713	0.1103
5	5 epochs lr: 0.01	0.9633	0.1220	0.9653	0.1215
6	20 epochs lr: 0.001	0.9610	0.1333	0.9715	0.1095
7	20 epochs lr: 0.0025	0.9725	0.0899	0.9722	0.0942
8	20 epochs lr: 0.005	0.9811	0.0610	0.9782	0.0763
9	20 epochs lr: 0.0075	0.9880	0.0384	0.9827	0.0650
10	20 epochs lr: 0.01	0.9936	0.0224	0.9853	0.0596

Conclusion:

Learning rate: 0.001:

Highest loss, lowest accuracy (not by much tho). It’s slow and requires more computational resources.

Learning rate: 0.0025:

Better metrics: loss under 0.1, accuracy over 97% for 20 epochs.

Learning rate: 0.005:

Good accuracy ~98% for 20 epochs. Great loss value: under 0.01.

Learning rate: 0.0075:

Achieves in 5 epoches almost the same accuracy of 0.001 withing 20 epochs. Loss is slowly reaching towards 0.05. Great performance.

Learning rate: 0.01:

Highest validation accuracy and lowest validation loss but the difference between training and validation accuracy = 0.83, which might be a sign of early overfitting.

Tendency: the higher the learning rate, the bigger the difference between the training and validation values.

Learning Rate Schedules

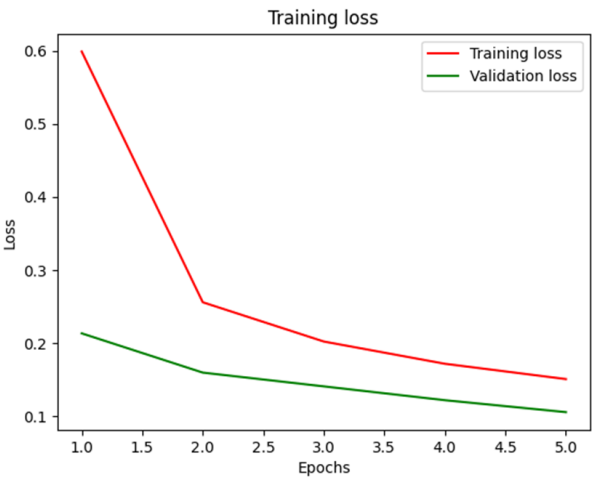
The learning rate is a parameter that controls how much to adjust the model parameters in response to the estimated error each time the model weights are updated during training.

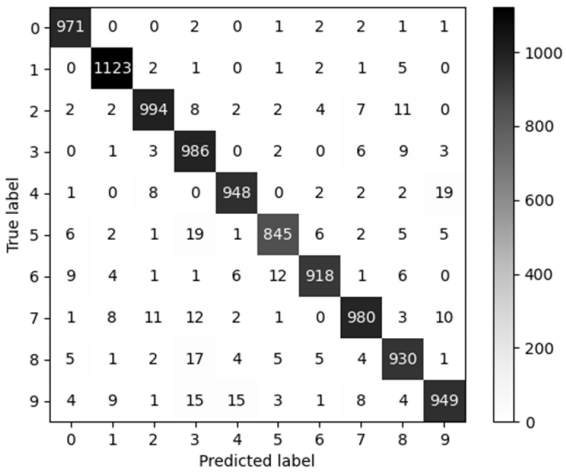
Initial LR	Final Train Acc	Final Val Acc	Final Train Loss	Final Val Loss	Training Speed	Val. Performance	Stability
0.001	95.47%	96.93%	0.1517	0.1162	Slow and Steady	Good	Very Stable
0.005	98.68%	98.65%	0.0424	0.0537	Fast	Excellent	Stable
0.01	99.06%	98.68%	0.0292	0.0494	Very Fast	Excellent	Stable
0.1	10.50%	10.50%	2.3020	2.3025	No learning	Failed	No Stability

Used the initial_learning_rate of 0.1, 0.01, 0.005, 0.001.

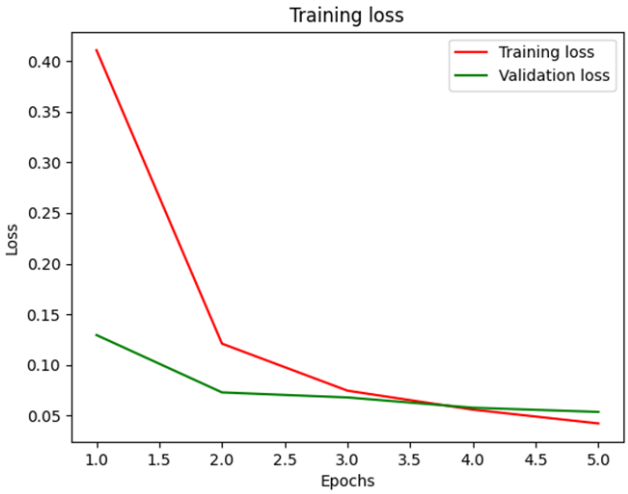
Conclusion:

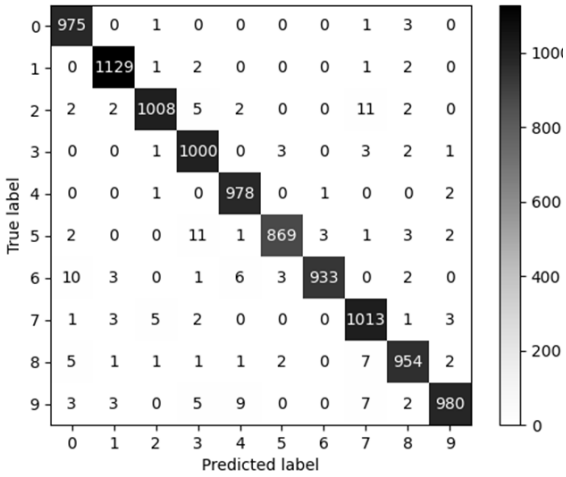
LR = 0.001 is very stable but slow



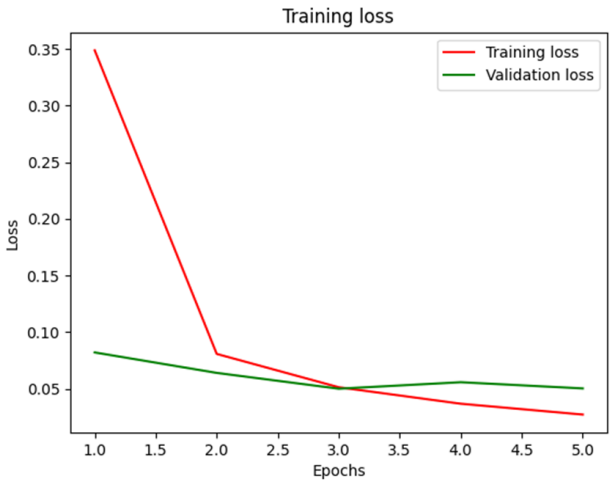


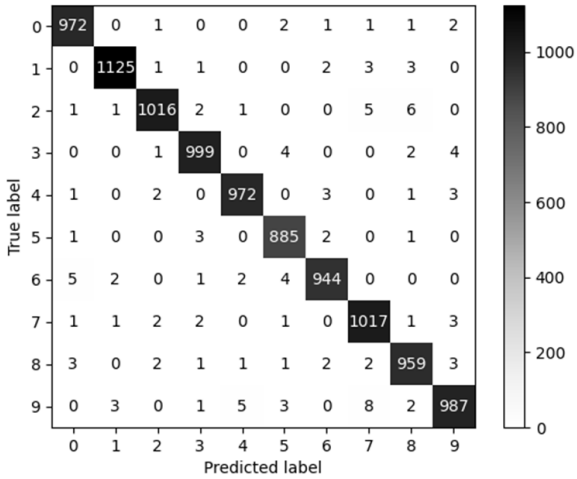
LR = 0.005 Has a good balance



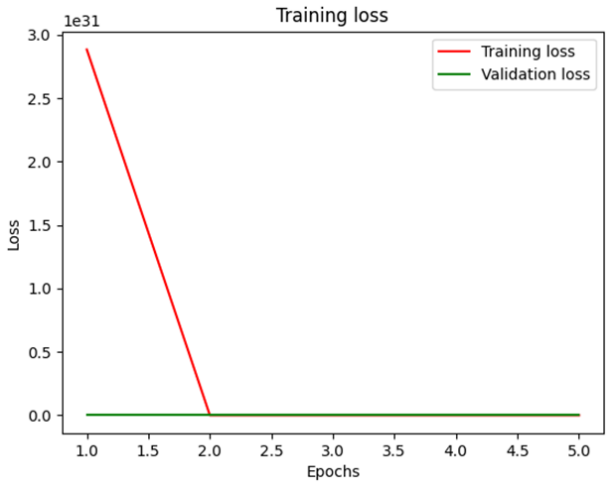


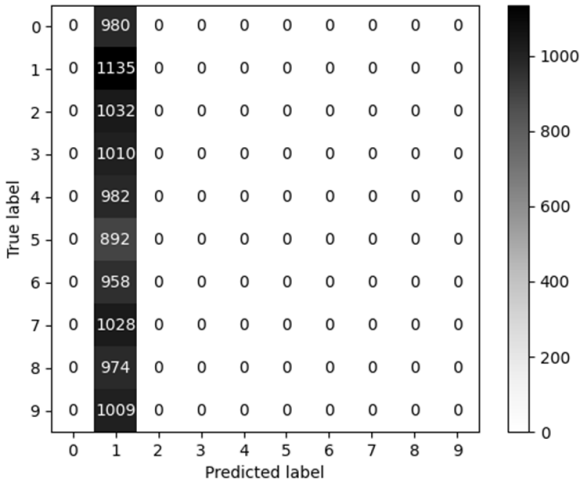
LR = 0.01 slightly overfits but still good numbers





LR = 0.1 guesses random not usable.


Training loss
1e31
Training loss
Validation loss
Loss
Epochs
3.0
2.5
2.0
1.5
1.0
0.5
0.0
1.0
1.5
2.0
2.5
3.0
3.5
4.0
4.5
5.0


True label
Predicted label
0 0 980 0 0 0 0 0 0 0 0
1 0 1135 0 0 0 0 0 0 0 0
2 0 1032 0 0 0 0 0 0 0 0
3 0 1010 0 0 0 0 0 0 0 0
4 0 982 0 0 0 0 0 0 0 0
5 0 892 0 0 0 0 0 0 0 0
6 0 958 0 0 0 0 0 0 0 0
7 0 1028 0 0 0 0 0 0 0 0
8 0 974 0 0 0 0 0 0 0 0
9 0 1009 0 0 0 0 0 0 0 0
0 1 2 3 4 5 6 7 8 9
1000
800
600
400
200
0

Optimizers

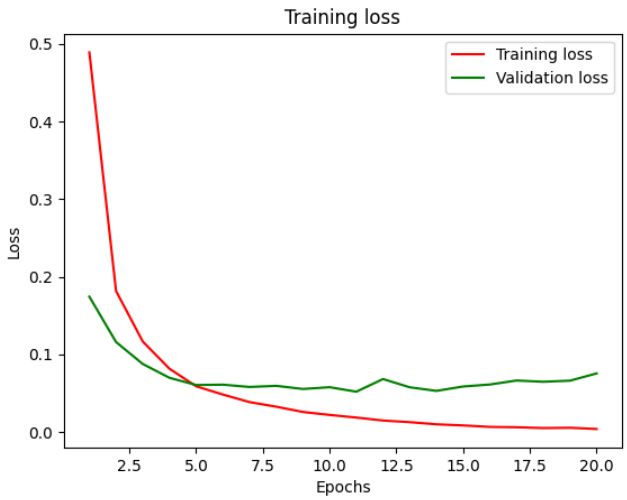
Momentum: SGD doesnt have to recalculate the steps from scratch every time. It can continue in the direction it was already moving to.

We changed the momentum to 0.9 and then tested a little bit using 4 different learning rates. Codewise there wasnt a lot to add:

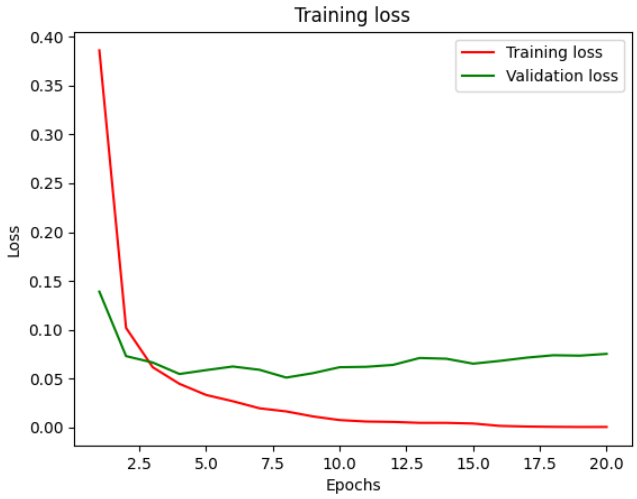
```
learning_rate = 0.01
momentum = 0.9
optimizer=SGD(learning_rate=learning_rate, momentum = 0.9)
model_name += '_LearningRate_' + str(learning_rate)
model_name += 'MomentumRate_' + str(momentum)
```

Learning Rate 0.001

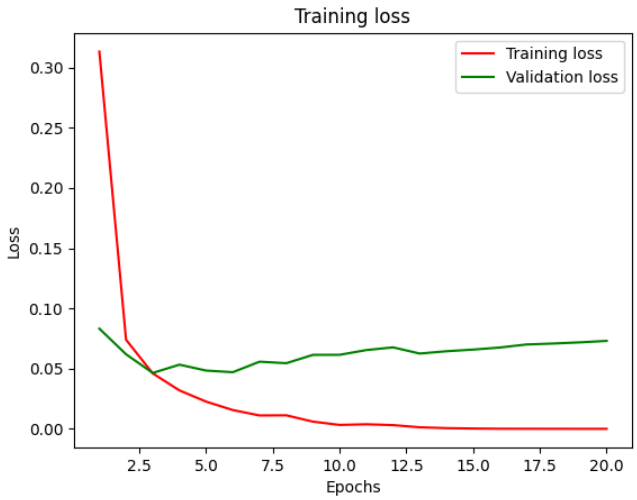
Learning Rate 0.0025



Learning Rate 0.005



Learning Rate 0.01



Final Values:

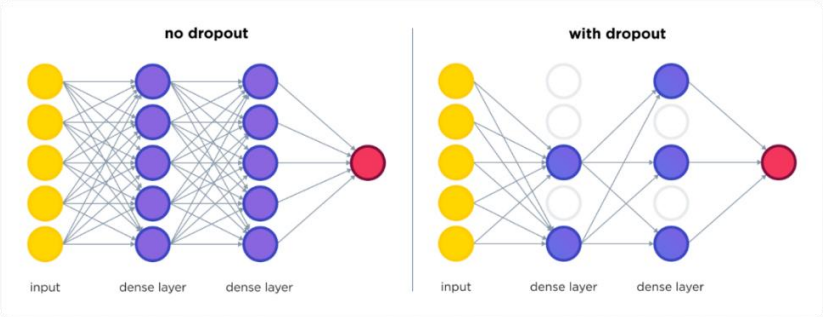
Learning Rate	Final Train Acc.	Final Value Acc.	Final Train Loss	Final Value Loss
0.001	99.3%	98.52%	0.0231	0.0609
0.0025	99.88%	98.43%	0.0042	0.0757
0.005	99.99%	98.80%	0.0004	0.0753
0.01	100%	98.97%	0.0005	0.0731

Conclusion:

As we can see in the values, no matter the learning rate, the training and the value accuracy are always 98%-100%. This doesn't mean they are all equally as good. If we compare the train and value loss, we can see for the 0.001 learning rate for example that the value loss is approximately 2.5-3 times as high as the train loss, which is a good indicator. For the 0.01 learning rate on the other hand, the value loss is around 140-150 times as high as the train loss, which means our model didn't really learn logical connections, but rather just memorize the data -> overfitting

Dropout Layer (Regularization)

Dropout is a technique where **randomly selected neurons** are **ignored** (dropped out) during training.



In this task, I analyse the values of the loss function for training and validation data, when

- there is dropout layer
- there is not.

Without dropout

In neural network, there is no dropout layers:

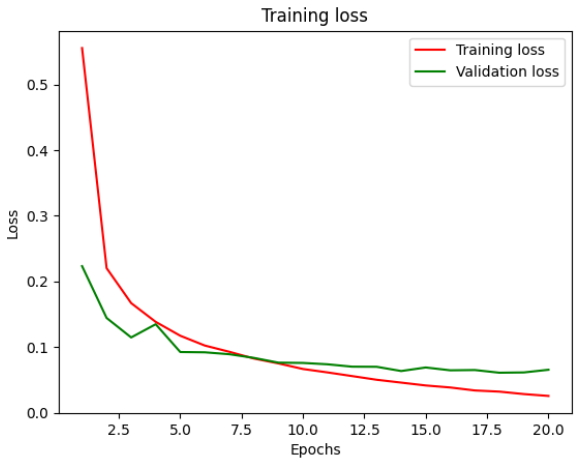
```
model = Sequential()
# convolutional layer
cnn1 = Conv2D(n_cnn1planes, kernel_size=(n_cnn1kernel,n_cnn1kernel), strides=(n_s
model.add(cnn1)
model.add(MaxPool2D(pool_size=(n_poolsize,n_poolsize)))

cnn2 = Conv2D(n_cnn1planes*2, kernel_size=(n_cnn1kernel,n_cnn1kernel), strides=(n
model.add(cnn2)
model.add(MaxPool2D(pool_size=(n_poolsize,n_poolsize)))


cnn3 = Conv2D(n_cnn1planes*4, kernel_size=(n_cnn1kernel,n_cnn1kernel), strides=(n
model.add(cnn3)
model.add(MaxPool2D(pool_size=(n_poolsize,n_poolsize)))


# flatten output of conv
model.add(Flatten())
```

With dropout

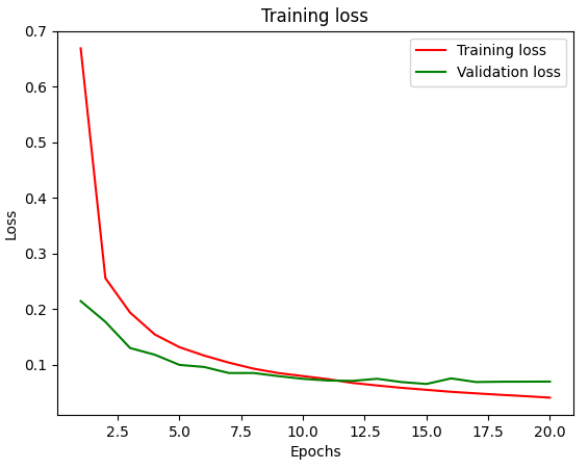
In neural network, there is dropout layer:

```
model = Sequential()
# convolutional layer
cnn1 = Conv2D(n_cnn1planes, kernel_size=(n_cnn1kernel,n_cnn1kernel), strides=(n_str
model.add(cnn1)
model.add(MaxPool2D(pool_size=(n_poolsize,n_poolsize)))

model.add(Dropout(dropout))

cnn2 = Conv2D(n_cnn1planes*2, kernel_size=(n_cnn1kernel,n_cnn1kernel), strides=(n_s
model.add(cnn2)
model.add(MaxPool2D(pool_size=(n_poolsize,n_poolsize)))

model.add(Dropout(dropout))

cnn3 = Conv2D(n_cnn1planes*4, kernel_size=(n_cnn1kernel,n_cnn1kernel), strides=(n_s
model.add(cnn3)
model.add(MaxPool2D(pool_size=(n_poolsize,n_poolsize)))

model.add(Dropout(dropout))

# flatten output of conv
model.add(Flatten())
```

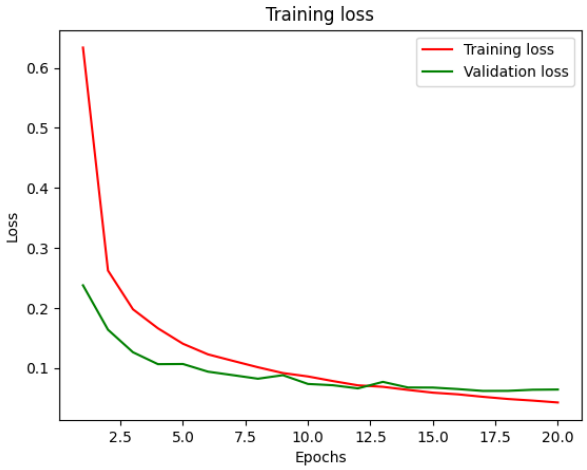
Comparing



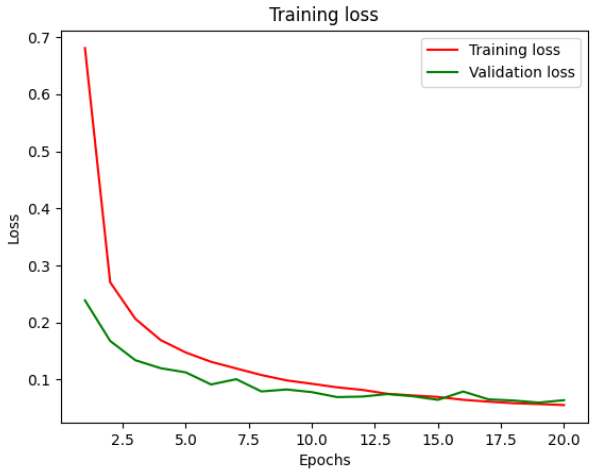
Without Dropout



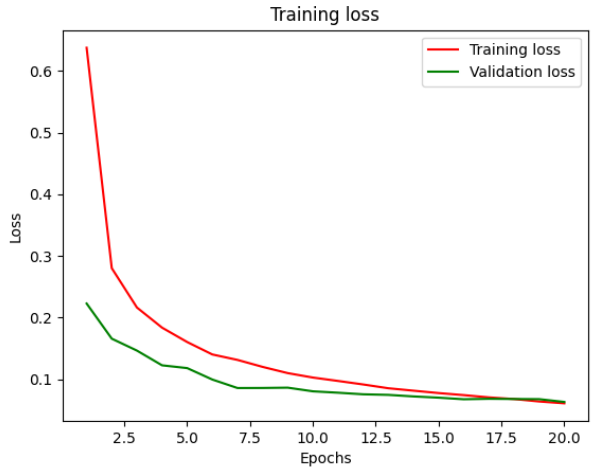
With Dropout **0.2**



With Dropout **0.3**



With Dropout **0.4**



With Dropout **0.5**

Conclusion

Without dropout there's larger gap between training and validation loss, but with dropout there is a smoother validation loss, and it reduces the overfitting in the model. A higher dropout rate slows the training but there is a better generalization and with a lower dropout rate the training is faster, but the potential of overfitting is high.

The optimal dropout rate is somewhere between 0.3 and 0.4.

Conclusion

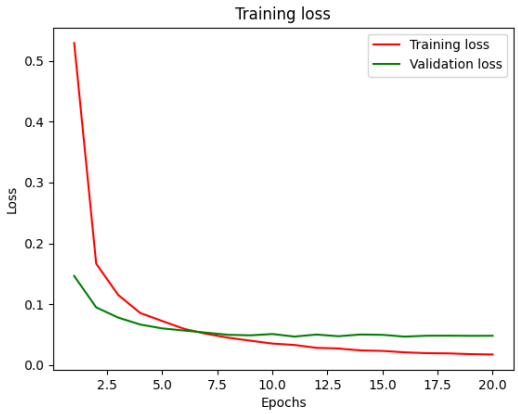
Planes: 20

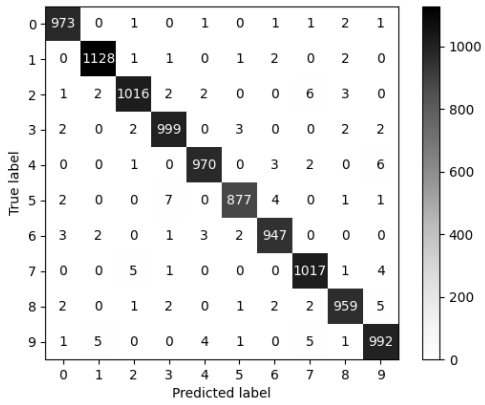
Epochs: 20

Learning rate: 0.005

Dropout rate: 0.3

```
422/422 ━━━━━━━━━━━━━━━━━━━━ 35s 84ms/step - accuracy: 0.9928 - loss: 0.0233 - val_accuracy: 0.9880 - val_loss: 0.0497
Epoch 16/20
422/422 ━━━━━━━━━━━━━━━━━━━━ 35s 83ms/step - accuracy: 0.9934 - loss: 0.0210 - val_accuracy: 0.9875 - val_loss: 0.0468
Epoch 17/20
422/422 ━━━━━━━━━━━━━━━━━━━━ 35s 83ms/step - accuracy: 0.9934 - loss: 0.0196 - val_accuracy: 0.9875 - val_loss: 0.0482
Epoch 18/20
422/422 ━━━━━━━━━━━━━━━━━━━━ 35s 83ms/step - accuracy: 0.9935 - loss: 0.0192 - val_accuracy: 0.9875 - val_loss: 0.0483
Epoch 19/20
422/422 ━━━━━━━━━━━━━━━━━━━━ 35s 83ms/step - accuracy: 0.9944 - loss: 0.0180 - val_accuracy: 0.9882 - val_loss: 0.0481
Epoch 20/20
422/422 ━━━━━━━━━━━━━━━━━━━━ 36s 85ms/step - accuracy: 0.9942 - loss: 0.0174 - val_accuracy: 0.9875 - val_loss: 0.0482
```





	precision	recall	f1-score	support
0	0.9888	0.9929	0.9908	980
1	0.9921	0.9938	0.9930	1135
2	0.9893	0.9845	0.9869	1032
3	0.9862	0.9891	0.9876	1010
4	0.9898	0.9878	0.9888	982
5	0.9910	0.9832	0.9871	892
6	0.9875	0.9885	0.9880	958
7	0.9845	0.9893	0.9869	1028
8	0.9876	0.9846	0.9861	974
9	0.9812	0.9832	0.9822	1009
accuracy			0.9878	10000
macro avg	0.9878	0.9877	0.9877	10000
weighted avg	0.9878	0.9878	0.9878	10000

Difference between validation and training loss is 0.03. Difference between validation and training accuracy is 0.6

Optimal configuration achieved: The combination of 20 filters, dropout 0.3, and learning rate 0.005 successfully balances model capacity with generalization.

Digit confusion patterns: Lower performance on digits 9, 8, and 2 suggests these share visual features that create ambiguity (curves, loops, similar stroke patterns).
Regularization effectiveness: 0.3 dropout rate prevents overfitting while maintaining 98.78% accuracy - proving the sweet spot between 0.5 (too aggressive) and no dropout.
Architecture sufficiency: 20 filters per layer provide adequate feature extraction without the computational overhead and minimal gains of 40 filters.
Training efficiency: Model converges smoothly within 20 epochs without requiring extended training or early stopping.